Putting it all together: Dot notation

Now that we're familiar with the basic ideas of Object Oriented Programming (OOP), namely Classes, properties, and methods(), let's discuss how to use them in code.

In OOP we use a convention called "dot notation" to access an object's properties or methods(). The "dot" of "dot notation" refers to the '.' symbol that separates an object and its property or method(). Consider the following example:

```
MyClass.property
MyClass.method()
```

We see that we need to reference an object, then use a "dot" to access its property or method. This is a very useful pattern, and we'll use it all of the time. So, to make sure we've got it, remember this basic patter:

```
MyClass.property
MyClass.method()
```

However, we will rarely call a property or method() on a class *directly*. Instead, we will create an *instance* of a class (think of this as a copy of a class) that we will then store in a variable. Then, we can use dot notation to access the instance's properties and methods() by replacing the above Class name with the variable that the instance is stored in. Here's an example:

```
# This line creates an instance of MyClass and stores it in the variable
my_object
var my_object = MyClass.new()

# We can then access the MyClass properties and methods() by using dot
notation
my_object.property
my_object.method()
```

Using Dot Notation

In this example we have a **Class** called **Desk**. The **Desk** class has the following properties and methods:

properties	description	methods	returns	description
height	The height of the desk in meters	move()	void	Moves the desk to an (x,y,z) coordinate
weight	The weight of the desk in kilograms	hold()	void	Places an object on the desk (the desk stores that object)
color	The color of the desk in ARGB	flip()	void	Flips the desk, removing all objects
legs	The amount of legs	rotate()	void	Rotates the desk around its y axis in radians
position	The position of the desk as (x,y,z) coordinates	take()	Object	Removes an object from the desk
		remove_legs()	int	Removes an amount of legs. Returns an int of the amount of legs remaining

In GDScript, we may create a new object in the following way:

```
var new_desk = Desk.new()
```

Remember, when we make an object we need to store it in a variable so that we can reference it later. Next, we may want to change some information about the desk. Remember, we can access an object's properties by using what we call dot notation. Here is an example accessing the legs property:

```
new_desk.legs
```

We can access any property in this way. Here are a few more examples:

```
new_desk.height
new_desk.color
new_desk.position
```

We can also change the value of a property by using an "=" to *assign* a new value to the property in the same way that we have with variables.

```
new_desk.legs = 3 # This desk now has 3 legs
```

```
new_desk.legs = 90 # This desk now has 90 legs
```

We can also use them in a calculation such as the example below:

```
var my_variable = new_desk.legs + 20 # This would store 110 in my_variable
```

We can access an objects methods() using dot notation as well. Below is an example of rotating the new_desk object:

```
new_desk.rotate(0.5 * PI) # This is in radians, in degrees this would be 90
degrees
```

And here we'll move the desk to some new location

```
# Notice that this method() takes 3 arguments separated by commas
new_desk.move(15, 0, 15)
```

You may have noticed that the Desk Class has a property for position that is an (x,y,z) coordinate as well. We could set the desk's position using either the move() method() or the position property.

```
# These two lines do the same thing
# A Vector3 collects an (x,y,z) coordinate together
new_desk.position = Vector3(15, 0, 15)
new_desk.move(15, 0, 15)
```

Some methods will *return* a value. This means that it will send *data* out when we call it. If we use the remove_legs() method we see that it will *return* an integer of the amount of legs remaining:

```

# This line would print whatever the current amount of legs minus 3 to the
output. Every time we ran this line the new_desk would have 3 less legs as
well.
print(new_desk.remove_legs(3))
```

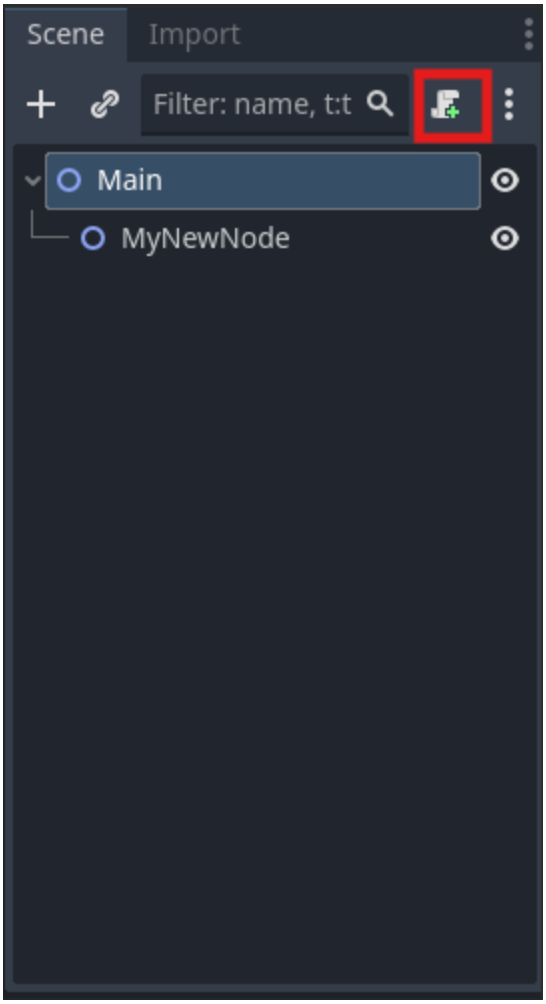
Working with Nodes and GDScript

Unlike other languages, we typically will not directly create instances of Classes in code directly (though we definitely can!). Instead, we will reference nodes that we've already created in the

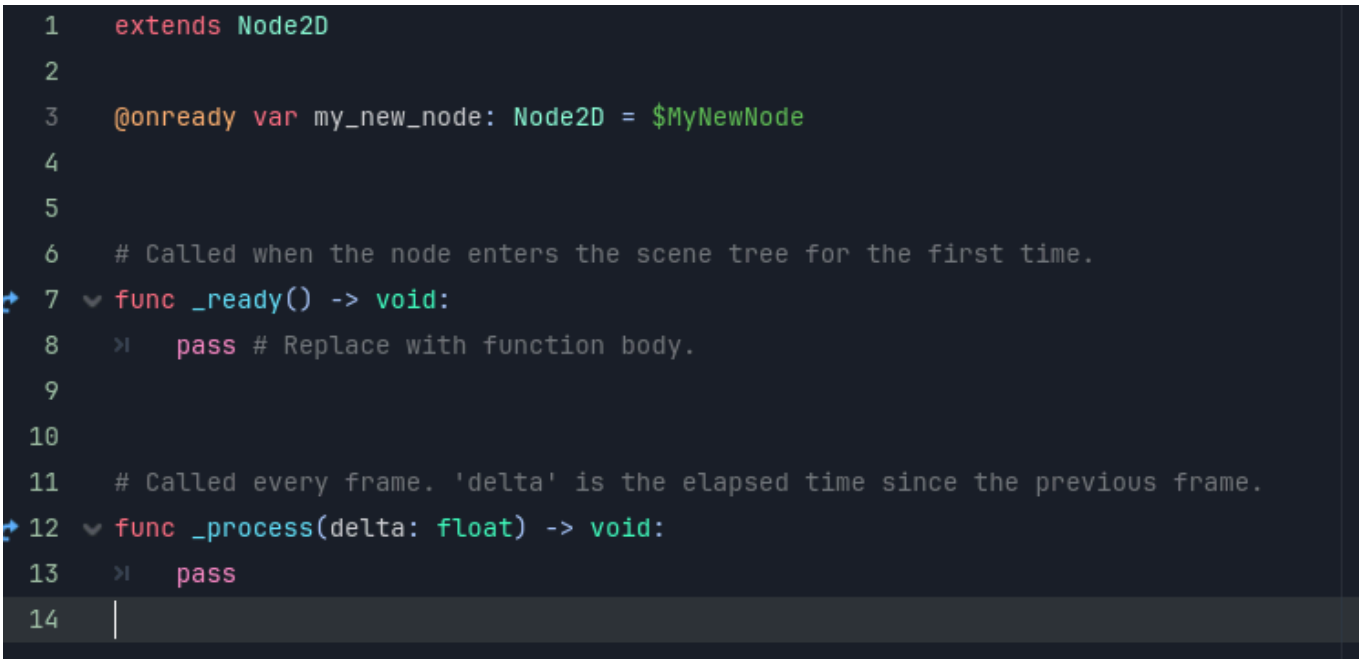
scene tree. We can do this in one of two ways: placing a reference to a node in a variable, or by attaching a script to a node you want to directly control. Let's start by going over how to get a reference into a script:

Option 1: Node References

First, we will need to attach a script to a node. We do this by selecting a node in the scene tree and pressing the "attach script" button found at the top of the scene tree dock (Note: if you do not select a node this button will not appear):



This will open our IDE (the script view). We can then drag a node from the scene tree into our script, hold the control key, and release the mouse to generate a variable that our node reference will be stored in. This will wind up looking like this:



Then we can use dot notation to access that node's properties and methods()! Make sure you use the correct variable name though, which is seen directly to the right of the "var" keyword. This is a good option for controlling multiple nodes from one script.

Option 2: Direct control

We can also access a node's properties and methods() *without* using dot notation if the script is attached directly to the node we want to manipulate. For instance, if we want to access a node's position property we would only need to write the following line:

```
func _ready():
    position
```

The above example is the same as the following example, except that the "self" keyword is used for the object the script is attached to:

```
func _ready():
    self.position
```

We can only manipulate the node that the script is *attached* to in this way. If we want to manipulate a different node that the script is not attached to we *must* bring in a node reference.

Now you try!

Please create the following nodes in a scene tree, pay close attention to the parent/child relationships:

Node2D (Name this "Main")

- Sprite2D
- Node2D (Name this "MyNewNode")

Then, using code, do the following:

1. Add node references for the Sprite2D to a script attached to Main
2. Attach a script to MyNewNode
3. In the Main script:
    - Using a property, move the Sprite2D 20 pixels every frame (think about which function this should go in and which property you might be able to change to move something. Remember, a Node also shares its parents properties and methods(). If you can't find an appropriate property in the Sprite2D documentation check its parent's documentation!)
    - Using a method, rotate the Sprite2D 0.5 * PI radians once when you first run your scene
    - Using a method, print the Rect2 of the Sprite2D once when you first run your scene.
4. In the MyNewNode script (these should all be done directly, ie no node references):
    - Using a property, print where the node is positioned once when the game is ran.
    - Using a method, move the node 20 pixels right along the x axis
    - Using a property, print the node's current rotation in degrees once when the game is ran.